Personal Expense & Income Tracking System

An Innovative Assignment for the subject of Software Engineering

Department Of Computer Science & Engineering

Veer Surendra Sai University Of Technology, Burla

Submitted by:

Name: Samuel Priyatam Dash

Registration no.: 2302040025

Semester: 6th

Subject name: Software Engineering(SE)

TABLE OF CONTENTS

1.1 Abstract..... 3
1.2 Introduction..... 3
Objectives..... 5
3. Methodology..... 6
Core Algorithms..... 6
Secure Stateless Authentication Model (JWT).....7
Asynchronous Object-Document Mapping (ODM)..... 7
Financial Aggregation and Synergy Pipeline.....8
System Architecture and Design..... 9
Technology Stack and Implementation Details..... 9
Technical Specifications.....10
Predictive Forecasting Implementation..... 11
System Workflow and Modules.....12
4. Results..... 13
User Interface and Simulation Scenarios..... 13
Scenario 1: High Risk (Security and Authentication Shock)..... 13
Scenario 2: Stable Project (Successful Financial Aggregation)..... 14
Scenario 3: Technical Debt Stress Test (Setup and Deployment).....14
5. Conclusion..... 15
6. Future Scope..... 15
7. References..... 16

1.1 Abstract

Modern personal financial management requires dynamic, real-time tracking architectures, as static techniques such as manual spreadsheet entry fail to capture the complexity of interacting socio-economic stressors. This report analyzes the architecture, algorithms, and empirical implementation of a minimal full-stack Personal Expense and Income Tracker. Built as a centralized decision-support tool, the system quantifies financial behaviors using advanced MongoDB aggregation pipelines, cryptographic session management, and asynchronous client-server communication.

Constructed on a robust technology stack featuring Python's FastAPI, MongoDB (Motor + Beanie ODM), and a React-based frontend utilizing Vite, TypeScript, and Recharts, the platform provides real-time, spatial mapping of personal financial stability. Research indicates that financial illiteracy and unmanaged personal expenditures cost the United States economy an estimated USD 246 billion annually, with over 51% of adults living paycheck to paycheck. The development of this system addresses these multi-dimensional challenges by integrating seamless expense tracking with graphical summaries, thereby reducing the cognitive load associated with financial planning. Furthermore, the development lifecycle of this project incorporated artificial intelligence methodologies to optimize code generation and architectural design; full context regarding the use of AI in this project is documented within the supplementary AI-Usage-Documentation.md file. This report details the system's architecture, setup parameters, RESTful API documentation, and empirical validation in transforming raw transactional metrics into heuristic, actionable financial guidance.

1.2 Introduction

Personal finance management is a continuous struggle against behavioral and economic entropy. Following the principles of increasing complexity, personal financial health degrades naturally unless active tracking, budgeting, and refactoring efforts maintain structural fiscal integrity. Modern consumer behavior often prioritizes rapid, frictionless consumption, leading to accumulated debt and the future cost of compounding interest necessitated by short-term spending habits. While systemic inflation and economic volatility are known external factors, their impact is magnified by organizational and behavioral stressors such as a lack of digital financial literacy.

Global trends indicate that the personal finance application market is expanding at a compound annual growth rate (CAGR) of 15.% to 20.%, projected to reach between USD 13. billion and USD 173. billion by 2035, driven by smartphone adoption and the transition toward digital banking. Despite this technological boom, existing tracking

solutions often suffer from systemic fragmentation, repetitive manual update requirements, and limited trust in the security of financial data. When high expense ratios coincide with a lack of emergency savings, individuals enter a state of financial vulnerability, avoiding critical economic corrections due to the cognitive load of disorganized data.

This system addresses these multi-dimensional challenges by treating financial risks as synergistic rather than isolated phenomena. It calculates "Financial Synergy," where overlapping stressors—such as high utility costs combined with variable income—create compounding failure probabilities in maintaining a personal budget. By deploying a minimal yet highly scalable full-stack application, the software provides a secure environment for categorizing transactions, viewing summaries, and managing incomes and expenses securely.

Metric Category	**Primary Risk Factor**	**Core Definition and Impact**
Technical	Fragmented Data Systems	The cognitive and temporal cost of manual data consolidation across multiple unlinked accounts, leading to reporting fatigue.
Behavioral	Impulsive Consumption	The rate at which unbudgeted expenditures erode savings, resulting in wealth loss and increased fiscal entropy.
External	Macroeconomic Volatility	The frequency of changes in living costs, inflation, and baseline utility requirements during the financial lifecycle.
Integrated	Risk Synergy	The compounding effect where overlapping financial liabilities (e.g., debt interest and income loss) create a magnified default probability.

Objectives

The primary objective is to provide a secure, integrated environment for identifying, categorizing, and mitigating personal financial risks through a robust web application. This is supported by several advanced technical goals designed to supersede legacy financial tracking models, leveraging the specific capabilities of FastAPI, MongoDB, and React.

Synergetic Analytical Engine: Traditional budget models often use linear addition, underestimating concurrent financial stressors. This system applies an aggregation methodology to reflect scenarios where high existing expenses make an individual exponentially more vulnerable to sudden income fluctuations. The backend provides an aggregated summary of transactions, allowing users to view total income, total expense, and net balance concurrently.

Spatial Data Visualizations: To move beyond raw numerical data and dense ledgers, the system uses Recharts and React Query to render interactive visual dashboards. These provide an immediate "at-a-glance" status of financial health, reducing cognitive overload and allowing for dynamic budget analysis based on categorical breakdowns.

Cryptographic Security and Heuristic Mitigation: Risk identification must lead to actionable intelligence securely. The application categorizes transactions within a heavily protected environment. All user endpoints require a valid JSON Web Token (JWT) Bearer token in the Authorization header, ensuring that heuristic mitigation strategies are delivered exclusively to authenticated entities without the risk of cross-site data leakage.

Predictive Deployment and Scalability: Applying modern software engineering principles, the system is designed for immediate concurrent deployment. By utilizing automated runner scripts (run.sh and run.ps1), the objective is to eliminate the friction typically associated with configuring full-stack environments, bridging the gap between development and production readiness.

Objective	Technical Approach	Expected Outcome
Quantification	MongoDB Aggregation Pipelines	Accurate numerical financial summaries, income-to-expense ratios, and categorical balances computed at the database layer.
Visualization	SVG Rendering via Recharts	Intuitive, real-time graphical representations of personal financial status mapped to the Document Object Model (DOM).
Security	Stateless PyJWT Authentication	Cryptographically verified session management ensuring user data isolation and tamper resistance using the HS256 algorithm.
Portability	Concurrent Execution Scripts	Rapid environment initialization via Bash and PowerShell scripts, bypassing complex manual staging configurations.
Forecasting	Continuous State Synchronization	Real-time UI updates powered by React Query, providing a constant assessment of current trajectory against historical norms.

3. Methodology

Core Algorithms

The analytical and operational foundation of the personal expense and income tracker rests on three distinct computational models: the Secure Stateless Authentication Model, the Asynchronous Object-Document Mapping structure, and the Financial Aggregation Pipeline. Each of these models serves to convert raw client-side inputs into secure, persistent, and quantifiable indicators of project and financial health.

Secure Stateless Authentication Model (JWT)

The security architecture defines authorization as a cryptographically signed payload rather than a stateful server-side session. To protect financial data, the system utilizes JSON Web Tokens (JWT) through the pyjwt library, adhering to RFC 7519 standards. The authentication propensity relies on the HMAC SHA-256 (HS256) algorithm, a symmetric cryptographic model that uses a single secret key for both the generation and validation of tokens.

The authentication algorithm calculates a secure token (T) as a concatenation of three Base64Url encoded strings: the Header (H), the Payload (P), and the Signature (S). The mathematical generation of the signature ensures that any adversarial modification to the payload invalidates the token completely:

$$S = HMACSHA256(Base64UrlEncode(H) + "." + Base64UrlEncode(P), Secret_Key)$$

When a user submits credentials via the OAuth2 form data structure to the /auth/login endpoint, the FastAPI backend evaluates the bcrypt-hashed password. Upon successful validation, the system generates the JWT containing claims such as the unique user identifier and expiration timestamp. This token is subsequently mandated in the Authorization header for all requests to the /transactions and /summary endpoints. This stateless design inherently scales across distributed systems while preventing traditional session hijacking and Cross-Site Request Forgery (CSRF) vectors.

Asynchronous Object-Document Mapping (ODM)

To quantify and persist the uncertainty introduced by varying financial inputs, the system utilizes Beanie ODM backed by the Motor asynchronous database engine. Beanie ODM integrates seamlessly with FastAPI, utilizing Pydantic models to enforce strict schema validation prior to database insertion.

The document model for a financial transaction considers multiple variables: a unique identifier, the associated user ID, the transaction type (income or expense), the monetary amount, currency (e.g., INR), categorical classification (e.g., Utilities, Food), description, and temporal timestamps (date, created_at, updated_at). The system posits that strictly typing these variables through Pydantic prevents malformed data from eroding the integrity of the financial ledger. Because Motor executes database operations asynchronously, the FastAPI event loop remains unblocked during heavy I/O operations, ensuring high throughput even during bulk transaction uploads.

Financial Aggregation and Synergy Pipeline

The core analytical innovation of the backend is its ability to account for categorical financial distributions natively within the database layer. Standard application-level calculations often retrieve all records and sum them in memory, which degrades performance synergistically as the dataset grows. However, this system relies on MongoDB's Aggregation Framework to calculate the financial summary factor natively.

The summary endpoint (GET /summary) triggers a multi-stage pipeline consisting of match, group, and project operators. The pipeline first filters documents belonging exclusively to the authenticated user, optionally restricted by a specific YYYY-MM temporal parameter. It then groups the documents by transaction type and category, applying the sum accumulator operator to aggregate the financial amounts.

The net balance calculation (B) is defined as the total accumulated income (I) minus the total accumulated expenses (E) across all categorized subsets (c):

$$B = \sum_{i=1}^{n} I_i - \sum_{c=1}^{k} E_c$$

This highly optimized pipeline allows the system to return an aggregated JSON object containing the total_income, total_expense, balance, and a nested category_breakdown dictionary. This structural efficiency ensures the frontend receives finalized analytical data, mitigating the need for complex, processing-heavy calculations on the client's device.

Algorithm / Mechanism	Primary Technical Tool	Application in System
Authentication	HS256 HMAC Cryptography	Generates tamper-proof JWT tokens for stateless API access.
Schema Validation	Pydantic & Beanie ODM	Enforces strict data types for financial transactions, preventing database entropy.
Financial Aggregation	MongoDB Pipeline	Calculates total balances and categorical distributions directly at the

Algorithm / Mechanism	Primary Technical Tool	Application in System
	Operators	database layer.
Data Fetching	React Query (TanStack)	Manages client-side background synchronization and caching of API responses.

System Architecture and Design

The system is built on a modular, decoupled architecture using Python for secure backend processing and React for data-driven front-end rendering. The architecture strictly separates the presentation layer from the data persistence and business logic layers.

Backend (FastAPI/Python): FastAPI serves as the high-performance orchestrator for data ingestion and algorithmic execution. Utilizing Python 3.+, the backend leverages the ASGI (Asynchronous Server Gateway Interface) specification via Uvicorn. The backend is responsible for verifying JWT claims, evaluating OAuth2 password request forms, and interfacing with the local MongoDB server.

Frontend (React/TypeScript): The interface binds data directly to the Document Object Model (DOM) using React components. Bootstrapped with Vite for optimized build processes and hot-module replacement, the frontend utilizes TypeScript to maintain type safety parallel to the backend's Pydantic schemas. Styling is achieved via TailwindCSS, providing a utility-first methodology for responsive design. The visualization modules heavily rely on Recharts, which parses the aggregated backend summary into interactive SVG elements, plotting expenses and incomes on Cartesian grids and pie charts.

Technology Stack and Implementation Details

The development of the system followed industry best practices for full-stack engineering, emphasizing an isolated local environment to prevent package collisions and ensure reproducible deployments across multiple operating systems.

The manual setup sequence demands specific prerequisites, notably Node.js (v18+), Python (v3.+), and a local MongoDB server instance (configurable via the MongoDB URI in backend/database.py). The manual deployment methodology requires the instantiation of a Python virtual environment.

Within the backend directory, administrators execute python3 -m venv venv followed by source activation. Dependencies are installed using the pip install command, specifically pulling fastapi, uvicorn[standard], pydantic[email], motor, beanie, passlib[bcrypt], bcrypt<4.., pyjwt, python-multipart, and standard testing libraries pytest and httpx. The ASGI server is then initialized on port 8000 using python -m uvicorn main:app --reload --port 8000. Concurrently, the frontend environment is prepared within the frontend directory through the execution of npm install and the subsequent launch of the development server via npm run dev.

Alternatively, a Quick Start methodology is provided to circumvent the manual provisioning process. Executable runner scripts are included to launch both the frontend and backend servers concurrently. On Linux or macOS environments, executing ./run.sh initializes the stack, whereas Windows environments utilize the PowerShell equivalent .\run.ps1. Upon execution, the graphical interface is exposed at http://localhost:5173, while the interactive Swagger API documentation is securely hosted at http://localhost:8000/docs.

Technical Specifications

The communication between the frontend client and the backend logic engine relies exclusively on asynchronous JSON payloads over HTTP. The backend provides a fully documented RESTful API. Every endpoint, with the exception of the authentication routes, enforces a strict dependency on the JWT Bearer token format.

The API documentation specifies distinct operational domains. The Authentication domain handles identity provisioning. The POST /auth/register endpoint accepts a JSON body containing an email and password, returning a confirmation message upon successful document creation. The POST /auth/login endpoint expects OAuth2-compliant form data (username and password), generating the signed JWT response containing the access_token and token_type.

The Transactions domain supports comprehensive state mutations. The GET /transactions endpoint retrieves an array of user-specific records, ordered chronologically, with an optional month query parameter (formatted as YYYY-MM) for temporal filtering.

API Endpoint	HTTP Method	Request Payload / Parameters	Expected Response Object
/transactions	POST	type, amount, currency, category, description, date (optional)	The fully instantiated transaction document, including the database-generated id.
/transactions/{id}	GET	Path Parameter: Transaction id	The specific transaction document; returns 404 if unauthorized or nonexistent.
/transactions/{id}	PUT	Partial JSON object containing fields to mutate	The updated transaction document reflecting the modified state.
/transactions/{id}	DELETE	Path Parameter: Transaction id	A confirmation message indicating successful document deletion.

The Summary domain exposes the GET /summary endpoint, returning the highly aggregated transaction statistics. Accepting the optional month query parameter, this endpoint yields a structured response detailing the total_income, total_expense, balance, and a nested category_breakdown mapping categories (e.g., Rent, Food) to their respective monetary totals.

Predictive Forecasting Implementation

While the current system architecture relies on empirical historical aggregation, the modular design accommodates predictive forecasting. A least-squares linear regression model can be applied over the time-series transaction data retrieved from the backend. The system calculates the trajectory of a user's expense habits by mapping the temporal sequence of transaction dates against the cumulative categorical amounts.

By calculating the slope (m) and intercept (b) of the trend line over a multi-month period, the application could mathematically project the anticipated depletion of the net balance. If the trajectory indicates a steep positive incline in the total_expense vector that surpasses the total_income vector, the visualization layer could trigger heuristic warnings, advising immediate financial mitigation and refactoring of discretionary spending categories.

System Workflow and Modules

The operational workflow of the system is designed to facilitate a continuous loop of financial assessment and budget correction. This cycle is divided into four distinct functional modules that act synergistically to maintain application state and user awareness.

Workflow Stage	**Primary Activity**	**System Component**
Input Module	Users provide financial data (amounts, categories, temporal data) via authenticated forms.	React Dashboard UI / TailwindCSS
Analytical Module	The engine authenticates requests, validates schemas, and processes aggregations.	FastAPI Logic Engine / Beanie ODM
Mapping Module	Numerical summaries are translated into interactive spatial graphics (e.g., categorical pie charts).	Recharts / React Query
Authorization	Cryptographic verification ensures data is strictly isolated and mathematically protected from tampering.	PyJWT / OAuth2 Password Bearer

4. Results

User Interface and Simulation Scenarios

The efficacy of the Personal Expense & Income Tracker was evaluated through a series of simulations designed to test the responsiveness of the logic engine, the security of the API endpoints, and the clarity of the visualization output. These scenarios represent common "stress tests" found in real-world software operation and personal finance management lifecycles.

Scenario 1: High Risk (Security and Authentication Shock)

An unauthorized entity attempts to manipulate the application state by fabricating a JSON Web Token to access another user's financial records via the GET /transactions endpoint.

- **Response:** The FastAPI dependency injection system extracts the token from the Authorization header. The PyJWT decoding algorithm attempts to verify the signature using the server's securely stored Secret_Key. Because the token was signed with an invalid or absent key, the cryptographic hash fails validation.
- **Output:** The backend immediately rejects the payload, returning an HTTP 401 Unauthorized exception. The React Query interceptor on the frontend catches this error, preventing any unauthorized rendering of data and redirecting the client to the login module. The system's reliance on stateless cryptography successfully nullifies the IDOR (Insecure Direct Object Reference) threat.

Security Metric	**Initial State**	**Scenario Action**	**Impact / Outcome**
Payload Identity	User A	Manipulated to User B	Header/Payload desynchronized from signature
Signature Validation	Valid HS256 Hash	Invalid Hash Evaluation	Cryptographic failure triggered

Security Metric	Initial State	Scenario Action	Impact / Outcome
API Access Level	200 OK	401 Unauthorized	Total denial of service to unauthorized entity
Data Leakage	Zero	Zero	Complete isolation of financial records maintained

Scenario 2: Stable Project (Successful Financial Aggregation)

A user logs a high frequency of micro-transactions (expenses) and a single large macro-transaction (income) over a continuous 30-day period. The user then navigates to the summary dashboard to view the current month's financial health.

- **Response:** The frontend dispatches a GET /summary?month=2023-10 request. The MongoDB aggregation pipeline filters thousands of records down to the specific month, groups them by transaction type, and calculates the summations natively within the database engine rather than transferring raw data to the Python runtime.
- **Output:** The API returns the aggregated JSON object in milliseconds. Recharts maps the category_breakdown to a pie chart, instantly visualizing that "Utilities" and "Food" consume the highest percentage of the income. The seamless interaction between React Query and the FastAPI backend ensures that the interface remains fluid and non-blocking, proving the viability of the asynchronous architecture.

Scenario 3: Technical Debt Stress Test (Setup and Deployment)

A deployment environment requires a rapid teardown and initialization of the entire stack due to an infrastructure migration. The development team must avoid the technical debt of executing complex, multi-stage manual provisioning steps.

- **Response:** The team utilizes the integrated run.sh script on a Linux host. The script simultaneously checks for the presence of the Python virtual environment, installs missing dependencies via pip, launches the Uvicorn server, installs NPM packages, and boots the Vite development server.

- **Output:** The entire application stack achieves operational readiness without manual intervention. The frontend binds to port 5173 and the backend binds to port 8000. This mitigates configuration drift and ensures that the application can scale across multiple development environments with zero downtime associated with environmental setup.

5. Conclusion

The architectural analysis validates that personal financial management tools benefit immensely from asynchronous, non-blocking software methodologies. By adopting a minimal full-stack approach utilizing FastAPI and MongoDB, the system processes transactional data rapidly while maintaining strict schema compliance via Beanie ODM. The integration of the MongoDB aggregation framework prevents the analytical engine from becoming CPU-bound during complex summary calculations, proving more efficient than iterative application-layer math.

Furthermore, the implementation of JWT HS256 stateless authentication ensures that sensitive financial data remains mathematically isolated, eliminating the overhead of traditional session stores. On the client side, the synergy between React Query and Recharts bridges the gap between complex numerical arrays and stakeholder comprehension. Transforming raw expense ledgers into immediate spatial visualizations reduces the cognitive burden of budgeting, effectively shifting personal finance from a reactive reporting task into a proactive, visually guided strategy. However, the analysis notes a known limitation in the user interface layer: the system currently relies on basic error toasts rather than robust, feature-rich toast libraries, which marginally reduces the heuristic feedback fidelity during transaction failures or network interruptions.

6. Future Scope

Enhanced User Experience Frameworks: Upgrading the frontend application to replace the basic error toasts with robust toast libraries (e.g., React Hot Toast or Sonner). This will improve the heuristic feedback loop, providing users with persistent, stackable notifications during rapid CRUD operations on the transactions ledger.

Automated Financial Ingestion: Utilizing secure API gateways (such as Plaid or open banking standards) to automatically extract and synchronize transaction data directly from financial institutions, thereby eliminating the friction of manual data entry and improving the accuracy of the financial summaries.

Machine Learning Categorization: Replacing manual category selection with a Natural Language Processing (NLP) or supervised classification model trained on merchant description data. This would allow the system to automatically route transactions into appropriate buckets (e.g., mapping a "Starbucks" description directly to the "Food" category), enhancing the predictive capabilities of the summary pipeline.

Expanded AI Integration Documentation: Continuing to evolve the platform based on the foundational artificial intelligence methodologies noted in the AI-Usage-Documentation.md. Future iterations will explore the deployment of Large Language Models (LLMs) to act as advisory agents, scanning the user's categorical breakdowns to offer personalized, text-based recommendations for debt reduction and expense optimization.

7. References

1. Ramsey Solutions: The State of Personal Finance in America Q4 2025. (2025). Insights into financial behavior, savings trends, and economic challenges. [https://www.ramseysolutions.com/budgeting/state-of-personal-finance]

2. NFEC (National Financial Educators Council): Financial Illiteracy Costs: Survey, Causes, Consequences. (2025). Analysis of economic impact and behavioral causes of financial illiteracy. [https://www.financialeducatorscouncil.org/financial-illiteracy-costs/]

3. Research Nester: Personal Finance Apps Market Size | Growth Trends 2035. (2025). Market growth analysis and future projections of financial applications. [https://www.researchnester.com/reports/personal-finance-apps-market/8243]

4. IEEE Xplore: Personal Finance Management and Prediction using ML Algorithms. (2025). Application of machine learning techniques in financial prediction systems. [https://ieeexplore.ieee.org/document/10816949/]

5. MongoDB Docs: Aggregation Pipeline - Database Manual. (2026). Core concepts and optimization techniques for data aggregation in MongoDB. [https://www.mongodb.com/docs/manual/core/aggregation-pipeline/]